Author: Andreas Traut
Date: 08.05.2020

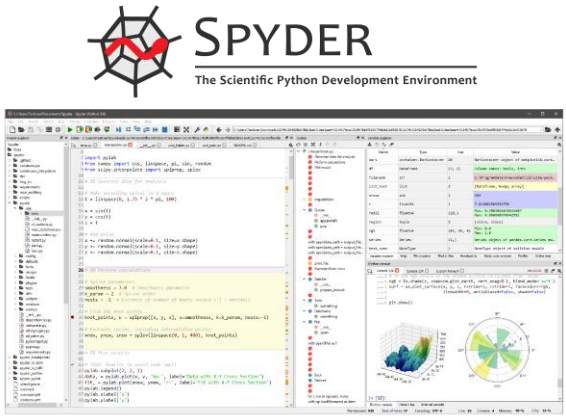
Machine Learning with Python

After having learnt visualization techniques in Python (which I showed in my repository "Visualization-of-Data-with-Python", see https://github.com/AndreasTraut/Visualization-of-Data-with-Python) I started working on different datasets with the aim to apply machine learning algorithms (e.g. Decision Tree to name only one).

In my examples in this repository here I will use Jupyter-Notebooks, which is a widespread standard today. The first Jupyter-Notebooks have been developed 5 years ago (in 2015). Since my first programming experience was more than 25 years ago (I started with GW-Basic then Turbo-Pascal and so on…) I quickly learnt the advantages of using Jupyter-Notebooks.

But I missed the comfort of an IDE from the very first days!

Therefore: in my examples in this repository here I will also work with Python ".py" files. These ".py" can be executed in an IDE, like e.g. Spyder-IDE (see https://www.spyder-ide.org/)



Spyder

Why is it important for me to point this out so early in a learning process?
In my opinion Jupyter-Notebooks are good for the first examinations of data and for documenting procedures and up to a certain degree also for sophisticated data science. But it might be a good idea to learn very early how to work with an IDE. Think about how to use what has been developed so far later in a bigger environment (for example a Lambda-Architecture, but you can take whatever other environment, which requires robustness&stability). I point this out here, because after having read several e-Books and having participated in seminars I see that IDEs are not in the focus.

Therefore the *first example* uses a Jupyter-Noteook in order to learn the standard procedures (e.g. data-cleaning&preparing, model-training,…).

The *second example* is for being used in an IDE. I will split it up into two parts: * 2.1 will use the "Scikit-Learn" python machine learning library * 2.2 will be an example for a "Big-Data" environment and uses the "Apache MLib" scalable machine learning library. Understanding the concept of "Big-Data" and how to differenciate "standard" machine learning from a "scalable" environment is not easy. I recommend a separate training.

1. Movies Database Example

A good starting point for finding useful datasets is "Kaggle" (www.kaggle.com). I downloaded the following movies dataset:

https://www.kaggle.com/isaactaylorofficial/imdb-10000-most-voted-feature-films-041118

The dataset from Kaggle contains the following columns:

Rank | Title | Year | Score | Metascore | Genre | Vote | Director | Runtime | **Revenue** | Description | RevCat

In this example I want to predict the **"Revenue"** based on the other information, which I have for each movie (e.g. every movie has a year, a scoring, a title …). There are some "NaN"-values in the column "Revenue" and instead of filling them with an assumption (e.g. median-value) as I did in another Jupiter-Notebook (see here), I wanted to predict these values. You might guess the conclusion already: predicting the revenue based on the available information as shown above (the columns) might not work. But essential to me is more to follow a well established standard-process of data-cleaning, data-preparing, model-training and error-calculation in this example in order to learn how to apply this process to better datasets, than the movies-dataset, later.

Therefore, here is how I approached the problem step-by-step: - I separated the rows with "NaN"-values in column "Revenue"

These are the datarows, where column "Revenue" is null:

```
In [10]: movies_RevenueNaN = movies[movies["Revenue"].isnull()]
         movies_RevenueNaN.head()
```

Out[10]:

	Rank	Title	Year	Score	Metascore	Genre	Vote	Director	Runtime	Revenue	Description
82	83	A Clockwork Orange	1971	8.3	80.0	Crime, Drama, Sci-Fi	662768	Stanley Kubrick	136	NaN	In the future, a sadistic gang leader is impri...
513	514	To Kill a Mockingbird	1962	8.3	87.0	Crime, Drama	262064	Robert Mulligan	129	NaN	Atticus Finch, a lawyer in the Depression-era ...
581	582	Death Proof	2007	7.0	NaN	Action, Thriller	236539	Quentin Tarantino	113	NaN	Two separate sets of voluptuous women are stal...
620	621	My Neighbour Totoro	1988	8.2	86.0	Animation, Family, Fantasy	226126	Hayao Miyazaki	86	NaN	When two girls move to the country to be near ...
685	686	Hachi: A Dog's Tale	2009	8.1	NaN	Drama, Family	212349	Lasse Hallström	93	NaN	A college professor's bond with the abandoned ...

```
In [11]: len_movies_RevenueNaN = len(movies_RevenueNaN)
         len_movies_RevenueNaN
```

Out[11]: 2527

- I drew a stratified sample (based on "Revenue") on this remaining dataset and I received a training dataset and testing dataset:

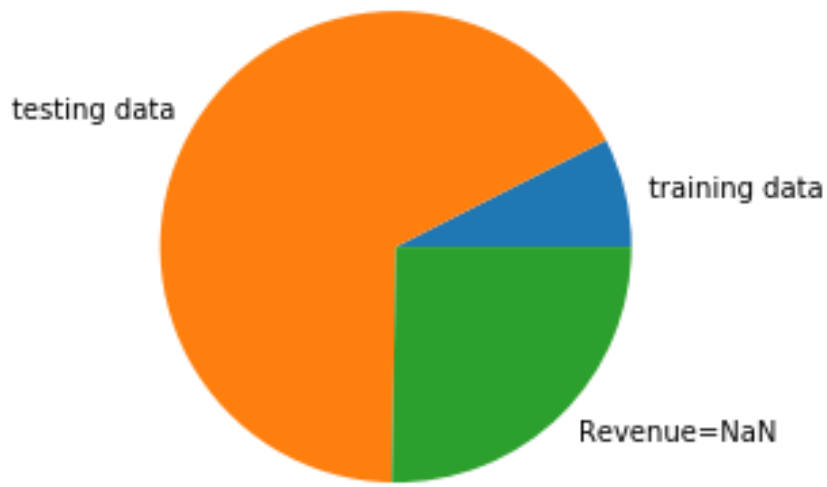
```
In [24]: split = StratifiedShuffleSplit(n_splits=1, test_size=0.9, random_state=42)
         for train_index, test_index in split.split(movies_NotNullC, movies_NotNullC["RevCat"]):
             strat_train_set = movies_NotNullC.iloc[train_index]
             strat_test_set = movies_NotNullC.iloc[test_index]
```

```
In [25]: strat_test_set["RevCat"].value_counts() / len(strat_test_set)

Out[25]: 1    0.903657
         2    0.069878
         3    0.016057
         4    0.010407
         Name: RevCat, dtype: float64
```

```
In [26]: movies_NotNullC["RevCat"].value_counts() / len(movies_NotNullC)

Out[26]: 1    0.903653
         2    0.069851
         3    0.016058
         4    0.010438
         Name: RevCat, dtype: float64
```



movies_train_test_nan

- I created a pipeline to fill the “NaN”-value in other columns (e.g. “Metascore”, “Score”).

```
In [38]: num_pipeline = Pipeline([
             ('imputer', SimpleImputer(strategy='median')),
         ])

         num_attribs = ['Rank', 'Year', 'Score', 'Metascore', 'Vote', 'Runtime']

         full_pipeline = ColumnTransformer([
             ('num', num_pipeline, num_attribs)
         ])
```

Now apply the Pipeline:

In [40]:
```
movies_train_prepared = full_pipeline.fit_transform(movies_train)
```

Now let's count the "nan" values in the new prepared datast "movies_train_prepared". I have to transform it back to a Pandas-Dataframe format first:

In [41]:
```
tmp_num = movies_train.select_dtypes(include=[np.number])
tmp_prep = pd.DataFrame(movies_train_prepared, columns=tmp_num.columns,index=movies_train.index)
tmp = tmp_prep[tmp_prep["Metascore"].isnull()]
tmp
```

Out[41]:

Rank	Year	Score	Metascore	Vote	Runtime

Zero, as we wanted! All "nan"-values in "movies_train_prepared" have been removed by the "median" value (this was how the pipeline was built). Great,

- used the training dataset and fittet it with the “DecisionTreeRegressor” model

In [42]:
```
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(movies_train_prepared, movies_train_labels)

movies_predictions = tree_reg.predict(movies_train_prepared)
```

- verified with a cross-validation, how good this model/parameters are

In [46]:
```
scores = cross_val_score(tree_reg,
                         movies_train_prepared,
                         movies_train_labels,
                         scoring="neg_mean_squared_error",
                         cv=10)

rmse_scores = np.sqrt(-scores)

def display_scores(scores):
    print("Scores:", scores)
    print("Mean:", scores.mean())
    print("Standard deviation:", scores.std())

display_scores(rmse_scores)
```

```
Scores: [ 69.66166984  62.59977724  62.6450061  112.98391756  47.99491086
  52.16787811  42.65104005  83.21059338  41.09199207  63.84158945]
Mean: 63.88483746577791
Standard deviation: 20.447326452253154
```

- did a prediction on a subset of the testing dataset and did a side-by-side comparison of prediction and true value

```
In [53]: side_by_side = [(true, pred) for true, pred in zip(list(some_data_label), list(some_data_predictions))]
         side_by_side

Out[53]: [(5.78, 10.91),
          (0.05, 0.44),
          (0.3, 2.68),
          (159.6, 11.99),
          (33.63, 2.19),
          (44.9, 26.83),
          (38.52, 22.52),
          (33.04, 2.19),
          (3.2, 11.99),
          (37.49, 16.38),
          (17.88, 64.19),
          (41.19, 191.45),
          (16.19, 12.19),
          (0.05, 3.02),
          (16.68, 64.19),
          (35.11, 11.54),
          (36.0, 40.22),
          (3.61, 19.64),
          (0.59, 0.05),
          (0.99, 5.48)]
```

- performed a prediction on the testing dataset and calculated the mean-squared error

```
In [55]: movies_test_prepared =full_pipeline.fit_transform(movies_test)
         movies_test_predictions = tree_reg.predict(movies_test_prepared)
         lin_mse = mean_squared_error(movies_test_labels, movies_test_predictions)
         lin_rmse = np.sqrt(lin_mse)
         lin_rmse

Out[55]: 54.68522284077671
```

The conclusion of this machine learning example is obvious: it is rather not possible to predict the "Revenue" based on the available information (the most useful numerical features were "year", "score", … and the other categorical like "genre" don't seem to have much more added value in my opinion).

Please find the complete Jupyter Notebook here:

https://github.com/AndreasTraut/Machine-Learning-with-Python/blob/master/Movies%20Machine%20Learning%20-%20Predict%20NaNs.ipynb

2. Step-by-step Python-Code for Machine Learning"

As Jupyter Notebooks are not always the best environment for Python coding (e.g. Debugging), I extracted the most essential parts of Chapter 2 of Aurelien Geron "Machine Learning with Scikit-Learn & Tensorflow", sorted and arranged the code fragments slightly and created the following structured Python code for being used for example in the Spyder-IDE. The structure of the Python code is a bit similar to the steps, which I followed in the Jupyter Notebooks above and are as follows (you will find these sections also in the ".py" file):

2.1 Using "scikit-learn"

1. create index
   1.1 Alternative 1: generate id with static data

```
# 1.1 Alternative 1: generate id with static data
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_train_test_by_id(housing_with_id, 0.2, "id")
```

1.2 Alternative 2: generate stratified sampling

```
# from sklearn.model_selection import StratifiedShuffleSplit
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]
```

1.3 verify if stratified example is good

```
compare_props["Rand. %error"] = 100 * compare_props["Random"] / compare_props["Overall"] - 100
compare_props["Strat. %error"] = 100 * compare_props["Stratified"] / compare_props["Overall"] - 100
```

2. Discover and visualize the data to gain insights

```
# from pandas.plotting import scatter_matrix
attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
plt.suptitle("scatter_matrix_plot")
save_fig("scatter_matrix_plot")
```

3. prepare for Machine Learning
   3.1 find all NULL-values

```
print("Are there nans in column total_bedrooms?\n", housing["total_bedrooms"].isnull().any())
print("Show rows with nan:\n", housing[housing["total_bedrooms"].isnull()])
```

3.2 remove all NULL-values

```
sample_incomplete_rows = housing[housing.isnull().any(axis=1)].head()
# sample_incomplete_rows.dropna(subset=["total_bedrooms"])    # option 1
# sample_incomplete_rows.drop("total_bedrooms", axis=1)       # option 2

median = housing["total_bedrooms"].median()
sample_incomplete_rows["total_bedrooms"].fillna(median, inplace=True) # option 3
```

4. Use “Imputer” to clean NaNs

```
imputer = SimpleImputer(strategy="median")
# Remove all text attributes because median can only be calculated on numerical attributes:
housing_num = housing.select_dtypes(include=[np.number])
imputer.fit(housing_num)
print("imputer.strategy\n", imputer.strategy)
print("imputer.statistics_\n", imputer.statistics_)
print("housing_num.median\n", housing_num.median().values)  # <- Check that this is the same as \
                                                             # manually computing the median of \
                                                             # each attribute
print("housing_num.mean\n", housing_num.mean().values)      # <- Check that this is the same as \
                                                             # manually computing the median of \
                                                             # each attribute
X = imputer.transform(housing_num) # Transform the training set:
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing.index)
```

5. treat “categorial” inputs

```python
cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

6. custom transformer and pipelines

6.1 custom transformer

```python
def add_extra_features(X, add_bedrooms_per_room=True):
    rooms_per_household = X[:, rooms_ix] / X[:, household_ix]
    population_per_household = X[:, population_ix] / X[:, household_ix]
    if add_bedrooms_per_room:
        bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
        return np.c_[X, rooms_per_household, population_per_household,
                     bedrooms_per_room]
    else:
        return np.c_[X, rooms_per_household, population_per_household]

# from sklearn.preprocessing import FunctionTransformer
attr_adder = FunctionTransformer(add_extra_features, validate=False,
                                 kw_args={"add_bedrooms_per_room": False})
housing_extra_attribs = attr_adder.fit_transform(housing.values)

housing_extra_attribs = pd.DataFrame(
    housing_extra_attribs,
    columns=list(housing.columns)+["rooms_per_household", "population_per_household"],
    index=housing.index)
print("housing_extra_attribs.head()\n", housing_extra_attribs.head())
```

6.2 pipelines

```python
num_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy="median")),
        ('attribs_adder', FunctionTransformer(add_extra_features,
                                              validate=False)),
        ('std_scaler', StandardScaler()),
    ])
housing_num_tr = num_pipeline.fit_transform(housing_num)
print("housing_num_tr\n", housing_num_tr)

try:
    from sklearn.compose import ColumnTransformer
except ImportError:
    from future_encoders import ColumnTransformer # Scikit-Learn < 0.20

num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
        ("num", num_pipeline, num_attribs),
        ("cat", OneHotEncoder(), cat_attribs),
    ])
housing_prepared = full_pipeline.fit_transform(housing)
print("housing_prepared\n", housing_prepared)
```

7. select and train model

7.1 LinearRegression model

```
# from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)
# let's try the full preprocessing pipeline on a few training instances
some_data = housing.iloc[:1]
some_labels = housing_labels.iloc[:1]
some_data_prepared = full_pipeline.transform(some_data)
print("Predictions:\n", lin_reg.predict(some_data_prepared))
print("Labels:\n", list(some_labels)) # Compare against the actual values:

# from sklearn.metrics import mean_squared_error
housing_predictions = lin_reg.predict(housing_prepared)
lin_mse = mean_squared_error(housing_labels, housing_predictions)
lin_rmse = np.sqrt(lin_mse)
print("lin_rmse\n", lin_rmse)
```

7.2 DecisionTreeRegressor model

```
# from sklearn.tree import DecisionTreeRegressor
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(housing_prepared, housing_labels)
housing_predictions = tree_reg.predict(housing_prepared)

tree_mse = mean_squared_error(housing_labels, housing_predictions)
tree_rmse = np.sqrt(tree_mse)
print("tree_rmse\n", tree_rmse)
```

8. crossvalidation

8.1 for DecisionTreeRegressor

```
# from sklearn.model_selection import cross_val_score
scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                         scoring="neg_mean_squared_error", cv=10)
```

8.2 for LinearRegression

```
lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                             scoring="neg_mean_squared_error", cv=10)
```

8.3 for RandomForestRegressor

```
# from sklearn.ensemble import RandomForestRegressor
forest_reg = RandomForestRegressor(n_estimators=10, random_state=42)
forest_reg.fit(housing_prepared, housing_labels)

housing_predictions = forest_reg.predict(housing_prepared)
forest_mse = mean_squared_error(housing_labels, housing_predictions)
forest_rmse = np.sqrt(forest_mse)
print(forest_rmse)
# from sklearn.model_selection import cross_val_score
forest_scores = cross_val_score(forest_reg, housing_prepared, housing_labels,
                                scoring="neg_mean_squared_error", cv=10)
forest_rmse_scores = np.sqrt(-forest_scores)
display_scores(forest_rmse_scores)
```

8.4 for ExtraTreesRegressor

```
from sklearn.ensemble import ExtraTreesRegressor
extratree_reg = ExtraTreesRegressor(n_estimators=10, random_state=42)
extratree_reg.fit(housing_prepared, housing_labels)

housing_predictions = extratree_reg.predict(housing_prepared)
extratree_mse = mean_squared_error(housing_labels, housing_predictions)
extratree_rmse = np.sqrt(extratree_mse)
print(extratree_rmse)
extratree_scores = cross_val_score(extratree_reg, housing_prepared,
                                   housing_labels,
                                   scoring = "neg_mean_squared_error", cv=10)
extratree_rmse_scores = np.sqrt(-extratree_scores)
display_scores(extratree_rmse_scores)
```

9. Save Model

```
# from sklearn.externals import joblib
joblib.dump(forest_reg, "forest_reg.pkl")
# und später zum Laden des Modells...
my_model_loaded = joblib.load("forest_reg.pkl")
```

10. Optimize Model
    10.1 GridSearchCV 10.1.1 GridSearchCV on RandomForestRegressor

```
# from sklearn.model_selection import GridSearchCV
param_grid = [
    # try 12 (3×4) combinations of hyperparameters
    {'n_estimators': [30, 40, 50], 'max_features': [2, 4, 6, 8, 10]},
    # then try 6 (2×3) combinations with bootstrap set as False
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
  ]

forest_reg = RandomForestRegressor(random_state=42)
# train across 5 folds, that's a total of (12+6)*5=90 rounds of training
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)
grid_search.fit(housing_prepared, housing_labels)
print(grid_search.best_params_)
print(grid_search.best_estimator_)
cvres = grid_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(np.sqrt(-mean_score), params)
```

10.1.2 GridSearchCV on LinearRegressor

```python
# from sklearn.model_selection import GridSearchCV
param_grid = [
    # try 12 (3×4) combinations of hyperparameters
    {'fit_intercept': [True], 'n_jobs': [2, 4, 6, 8, 10]},
    # then try 6 (2×3) combinations with bootstrap set as False
    {'normalize': [False], 'n_jobs': [3, 10]},
  ]

lin_reg = LinearRegression()
# train across 5 folds, that's a total of (12+6)*5=90 rounds of training
lin_grid_search = GridSearchCV(lin_reg, param_grid, cv=5,
                               scoring='neg_mean_squared_error',
                               return_train_score=True)
lin_grid_search.fit(housing_prepared, housing_labels)
# print(lin_grid_search.best_params_)
print(lin_grid_search.best_estimator_)
cvres = lin_grid_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(np.sqrt(-mean_score), params)
```

10.2 Randomized Search

```python
# from sklearn.model_selection import RandomizedSearchCV
# from scipy.stats import randint
param_distribs = {
        'n_estimators': randint(low=1, high=200),
        'max_features': randint(low=1, high=8),
    }

forest_reg = RandomForestRegressor(random_state=42)
rnd_search = RandomizedSearchCV(forest_reg,
                                param_distributions=param_distribs,
                                n_iter=10, cv=5,
                                scoring='neg_mean_squared_error',
                                random_state=42)
rnd_search.fit(housing_prepared, housing_labels)
cvres = rnd_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(np.sqrt(-mean_score), params)
```

10.3 Analyze best models

```python
feature_importances = grid_search.best_estimator_.feature_importances_
feature_importances
extra_attribs = ["rooms_per_hhold", "pop_per_hhold", "bedrooms_per_room"]
cat_encoder = full_pipeline.named_transformers_["cat"]
cat_one_hot_attribs = list(cat_encoder.categories_[0])
attributes = num_attribs + extra_attribs + cat_one_hot_attribs
sorted(zip(feature_importances, attributes), reverse=True)
```

11. Evaluate final model on test dataset

```python
final_model = grid_search.best_estimator_

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test)
final_predictions = final_model.predict(X_test_prepared)

final_mse = mean_squared_error(y_test, final_predictions)
final_rmse = np.sqrt(final_mse)

print ("final_predictions\n", final_predictions )
print ("final_rmse \n", final_rmse )

confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
mean = squared_errors.mean()
m = len(squared_errors)

# from scipy import stats
print("95% confidence interval: ",
      np.sqrt(stats.t.interval(confidence, m - 1,
                               loc=np.mean(squared_errors),
                               scale=stats.sem(squared_errors)))
      )
```

2.2 Using "Apache Machine-Learning" Libary (Big Data)

The steps are a bit similar (e.g. data-cleaning, preprocessing), to "scikit-learn" but the code is different:

0. Initialize Spark
   0.1 Create Spark Context and Spark Session 0.2 Read CSV 0.3 Dataset Properties and some Select, Group and Aggregate Methods 0.4 Write as Parquet or CSV 0.5 Read Parquet 0.6 How to stop a Spark Session and Spark Context

1. Cleaning the data
   1.1 Show number of rows and columns and do some visualizations 1.2 Replacing and Casting 1.3 Null-Values 1.4 String Values

2. Model-specific preprocessing
   2.0 Check missing entries and define userdefined scatter plot 2.1 StringIndexer 2.2 OntHotEncoder 2.3 VectorAssembler 2.4 CountVectorizer

3. Aligning and numerating Features and Labels
   3.1 Aligning 3.2 Numerating

4. Pipelines

5. Training data and Testing data

6. Apply models and evaluate
   6.1 Ordinary Least Square Regression 6.2 Ridge Regression 6.3 Lasso Regression 6.4 Decision Tree

7. Minhash und Local-Sensitive-Hashing (LSH)

8. Alternative-Least-Square (ALS)
   8.1. Datapreparation for ALS 8.2 Build the recommendation model using alternating least squares (ALS) 8.3 Get recommendations 8.4 Clustering of Users with K-Means 8.5 Perform a PCA and draw the 2-dim projection